COMPUTER SCIENCE + COMPUTER ENGINEERING

Gate D: a Defensible Systems Application

ForgeQueue / project specification

A local, bounded native workload service with validation, transactional replay records and failure recovery.

Requirements · Ownership · Failure matrix · Review rubric

Package 05 · Expanded Volume 04 · September 28, 2026
Self-study teaching edition. Completion requires independent implementation, testing and review.

ForgeQueue — worked local workload-processing service

The original checkpoint

The source program asks the learner to build a local job-based application that reads validated work, performs bounded processing and stores results. A small local client/server or task-processing tool is enough; public deployment is unnecessary. Required evidence includes a requirements sheet, architectural decision record, thread/resource ownership map, protocol/schema, threat model, automated build/test script, dependency inventory and recovery report.

The source failure cases are malformed messages, duplicate requests, cancellation, partial writes, worker failure and shutdown under load. The learner must explain data-race freedom under the stated model, avoidance of duplicated durable effects and evidence for the measured bottleneck. A happy-path demonstration alone does not pass.

ForgeQueue is a new worked realization of those requirements, not a replacement for the original checkpoint and not a completed student submission. A file-driven local tool is also acceptable when it provides the same validation, bounded work, persistence and failure evidence.

Scope and non-goals

The reference accepts a small versioned request, performs a bounded sum-of-squares operation in a trusted native executable and records the result transactionally. The values can stand for a synthetic per-entity workload. This is a systems-learning exercise, not a production asset compiler, game server or commercial backend.

No request can select an executable, filesystem path, shell command, worker count or arbitrary script. The service does not provide public authentication, encryption, distributed transactions, hostile-native-code isolation or a power-loss durability proof. Those capabilities cannot be inferred from a localhost test.

The native queue and finite task-graph labs remain required. The reference service uses Python threads to supervise bounded native subprocesses; it is not a demonstration of parallel Python numerical execution or a substitute for the C++ memory-model lessons.

Protocol contract

One gateway connection contains one length-prefixed request and one response, then closes. Extra pipelined requests on that connection are not supported. The parser exercise separately supports multiple frames in a stream; do not confuse that capability with the narrower gateway policy.

Field	Version-1 rule
Frame prefix	Four-byte unsigned big-endian body length.
Encoded body	1–4096 bytes of UTF-8 JSON.
version	Exact integer 1, not boolean or floating-point 1.0.
id	1–32 ASCII letters, digits, underscore or hyphen.

Field	Version-1 rule
operation	Exactly `sum_squares`.
values	List of 1–256 exact integers in 0–1000.
Other fields	Rejected. Duplicate JSON field names are rejected.

Example request:

```
{"version":1,"id":"demo-1","operation":"sum_squares","values":[2,3,4]}
```

Expected successful response:

```
{"version":1,"id":"demo-1","status":"done","result":29}
```

Maximum arithmetic result is `256 * 1000 * 1000 = 256000000`. The native implementation accumulates in an unsigned 64-bit type. The supervisor checks the result against an independent small Python reference before persistence. That duplicate work is affordable here; it is not a scalable validation strategy for every future engine workload.

Requirements and acceptance evidence

ID	Requirement	Minimum evidence
D-01	Reject malformed, oversized or unsupported input before native work.	Header, schema, range, duplicate-key and nesting tests.
D-02	Bound accepted queued-plus-running jobs.	Saturation with controlled worker events; next admission rejected.
D-03	Produce the defined arithmetic result or a distinct failure.	Native/reference comparison, empty/ limit rejection, abnormal-exit test.
D-04	Commit one result per ID and canonical payload.	Ordinary repeat, concurrent repeat, restart and conflicting-ID tests.
D-05	Define graceful drain and cancellation separately.	Pending/running cancellation and drain-under-load evidence.
D-06	Preserve authoritative state during specified failures.	Precommit rollback, postcommit lost-response replay and save-injection results.
D-07	Return useful textual status without leaking entire input or private paths.	Operator-message review and a second-person recovery exercise.
D-08	Build and verify from a clean source release.	Fresh build log, deliberately failed pipeline and extraction/rebuild.

ID	Requirement	Minimum evidence
D-09	Report performance without invented universal targets.	Raw workload trials, environment, timing boundary and bottleneck argument.
D-10	State residual risks and attribution.	Threat model, dependency inventory, source/reference attribution and limitations.

These are concrete assessment choices for this expansion. They elaborate the source Gate D requirements rather than adding a requirement to deploy publicly.

Resource budget

The reference defaults to two worker threads and four admission tickets. Configuration requires `1 <= workers <= capacity <= 32`. A ticket covers a queued or running job; the future's completion callback releases it. The supervisor may briefly run duplicate computations, but only one durable result is committed for identical ID/payload work.

Resource	Reference bound or assumption
Request body	4096 wire bytes before parsing.
Incremental decoder input chunk	8192 bytes per feed; incomplete body separately bounded.
Native input	At most 256 validated small integers.
Queued + running jobs	Capacity tickets; default 4.
Native subprocesses	At most the active worker count.
Native wait	5 seconds before the supervisor's timeout handling. Not a hard-real-time guarantee.
Frame receive	One absolute 1-second budget in the gateway.
Gateway result wait	2 seconds; expiry reports UNKNOWN_OUTCOME, not rollback.
New unique result records	At most 1024, or a smaller configured limit. Existing history is retained.
Connections handled in application	One active peer in the serial acceptor; kernel backlog behavior is not an authentication or DoS guarantee.

These are application-level bounds, not an assertion that total process resident memory equals wire bytes. Interpreter overhead, completed futures retained by callers, SQLite storage and operating-system buffering are separate. The selected trusted native worker emits one small integer; an arbitrary executable that floods output is outside the supported model.

Ownership and synchronization

Resource	Owner and lifetime
Raw request bytes	Caller until parsing; bounded by the framing interface.
Validated job dictionary	One submitted worker; no external mutation after parsing.
Admission ticket	Acquired before submission; returned once by completion callback, including cancelled futures.
Native process	Supervising worker; command is an argument list with no shell; wait/cleanup follows the subprocess API.
Result-store connection	The thread performing that operation; closed afterward, not shared across threads.
Service acceptance/cancel state	Protected by the service mutex.
Commit-versus-cancel decision	Serialized by that same service mutex around the store call.
Gateway listener	Gateway owner; stop request closes listener and joins the acceptor before service teardown.

The computation runs outside the service lock. Persistence takes the lock so that cancellation and committing have a defined order. Joining the worker pool happens after releasing the lock. The future callback only releases a semaphore; it must not take a lock held when registering a callback, because registration can call it immediately for an already completed future.

For the native graph, a predecessor writes its output before publishing terminal state under the graph mutex. A dependent claims work after observing that state through the mutex protocol. This does not authorize arbitrary unsynchronized writes between independent tasks; captured data still needs disjoint writes or its own synchronization.

Lifecycle and commitment

Conceptually, a job moves through `received -> validated -> admitted -> queued/running -> computed -> committed -> replied`. Rejection can occur before admission; worker failure can occur before commitment; a reply may be lost after commitment. The reference does not persist every transient lifecycle state. Its authoritative table contains committed results only.

Cancellation is cooperative at the service boundary. A pending future can be cancelled before execution. Running native work is allowed to return or hit its bounded supervisor timeout. Before persistence, the worker checks cancellation under the commit lock. If cancellation wins that boundary, no result is committed. If the commit wins first, later cancellation does not erase it.

A gateway timeout does not automatically request cancellation. The operation may continue and commit; the response explicitly instructs the caller to retry the same identity and payload. Do not introduce a new ID simply to make a timeout disappear.

Database and replay semantics

The authoritative result table stores ID, canonical validated request text and integer result. The ID is the primary key. A transaction checks for an existing row, rejects conflicting work or inserts the new row, then commits. The canonical text is compared directly; the design does not depend on treating a hash as collision-free identity.

SQLite is configured through explicit SQL transactions, with a bounded database wait and FULL synchronization setting in the reference. These settings are part of the observed lab configuration, not a universal guarantee against every storage or power failure. The inventory-transfer exercise demonstrates a stronger multi-row invariant in the same database family; it is related evidence, not a hidden second service transaction.

History retention is part of correctness. Once full, the reference rejects new unique work and retains records needed for replays. A production design might have a documented retention window and external client guarantees, but silently deleting old rows changes this project's guarantee. Restart means reopening the same retained database, not creating a fresh database with no history.

Required failure matrix

Case	Required observable result
Zero or over-limit frame length	Classified rejection before body allocation/native work.
Partial header/body then EOF	Truncation, no committed work.
Slow incomplete frame	Absolute deadline reached; connection closed or error returned.
Wrong version or unknown fields	Validation rejection, no native work.
Boolean/floating/negative/out-of-range values	Schema rejection.
Duplicate JSON keys	Rejection, not ambiguous last-key-wins interpretation.
Capacity exhausted	New work rejected while accepted jobs retain their contracts.
Worker abnormal exit/invalid result	Failure, no result row.
Same ID/same work twice	Same committed response, one row.
Concurrent same-ID work	One durable record; no contradictory committed result.
Same ID/different work	Conflict; original record unchanged.
Failure after transfer debit	Debit, credit and operation record roll back together.
Reply lost after commit	Retried original ID returns the committed result.
Cancel before commitment	No new result for cancelled work under the stated ordering.
Cancel after commitment	Previously committed result remains.

Case	Required observable result
Graceful shutdown under load	No new admissions; accepted work drains; users join.
History quota reached	Existing replay protection retained; new unique work rejected.
Partial temporary save write	Last destination remains valid under the injection model.
Failure after save replacement	New valid save visible despite response failure.

Control concurrency with events, futures or other defined synchronization when testing. Do not rely solely on lucky sleep durations to make a race appear. Tests must have external time limits so a shutdown defect fails the test rather than hanging forever.

Student milestones

Milestone 1 — Specification: Write the requirements, protocol, database schema and resource bounds. Review two architecture decisions before adding concurrency.

Milestone 2 — Serial vertical slice: Validate a request, execute one bounded operation and persist a checked result. Add ordinary and malformed cases.

Milestone 3 — Bounded work: Introduce admission limits and workers. Demonstrate saturation, errors and completion accounting. Complete the native queue/graph evidence.

Milestone 4 — Recovery: Add replay identity, conflicts, cancellation and shutdown. Test before/after commitment and restart against retained data.

Milestone 5 — Handoff: Complete measurements, threat review, source release, independent rebuild and live assessment. Add one student-owned extension rather than renaming reference variables.

A suitable independent extension is a second bounded game-data operation or a supported version-1-to-version-2 migration. Define its new limits, preserve old guarantees and test a failure case not present in the reference. Arbitrary script execution or public deployment is not a suitable shortcut to “advanced” scope.

Submission and scoring

Deliver source and exact run/test commands; requirements and traceability; two decision records; ownership/lifecycle documentation; protocol and schema; threat model; at least twenty learner-authored acceptance tests; failure/recovery evidence; raw performance data and analysis; dependency/attribution records; operator review; and a versioned handoff with limitations.

Criterion	Points
Correctness and requirements	30
Tests and verification	25
Reasoning and design tradeoffs	15
Quantitative performance or error analysis	10

Criterion	Points
Reproducibility	10
Communication and provenance	10

Suggested pass is **80/100**, all essential failure checks completed, and no unresolved critical defect within the claimed operating conditions. Do not award a pass merely because supplied reference tests pass. An unbounded queue, ambiguous duplicate-effect contract or deadlocked shutdown blocks the relevant claim regardless of presentation quality.

Oral review

Explain the complete path from received bytes to a committed row. Identify a memory/lifetime guarantee in native code. Explain why the queue wakes correctly at shutdown. Show the exact ordering between cancellation and commitment. Reproduce a lost-response retry without duplicating the durable effect. Defend a measured bottleneck and a limitation. Then implement a small changed requirement without copying a reference walkthrough.

A reviewer should be able to reproduce the central result from an extracted source tree. Completion is a self-study engineering milestone, not an accredited degree or a production-security certification.